👤 **Tester:** Paula Granado

🎯 **API Test Objective:**
Validate the integrity and functionality of the Swagger Petstore API endpoints, which manage pets, store inventory, and user accounts. The tests ensure data consistency, correct HTTP status codes, and proper handling of both valid and invalid requests.

🔒 **API Test Prerequisites**
- The Swagger Petstore API service must be running and accessible.
- A valid API key or authentication token must be available for authenticated endpoints.
- The test environment must have a stable connection.

🧪 **API Test Endpoints**

- POST /pet (Create a new pet to the store)
- GET /pet/{id} (Find pet by ID)
- GET /pet/findByStatus (Find pets by status)
- PUT /pet (Update an existing pet)
- DELETE /pet/{id} (Deletes a pet)
- GET /store/inventory (Returns pet inventories by status)
- POST /store/order (Place an order for a pet)
- POST /user (Create user)
- GET /user/login (Logs user into the system)

👥 **Test Scenarios & Profiles**
- Standard User: Valid API key with full access to product, cart, and checkout endpoints.
- Unauthorized User: No API key or an invalid key; should be rejected with a 401 Unauthorized status code.
- Locked-Out User: API key for a disabled user account; should be rejected with a 403 Forbidden status code.

🛠️ **Tools Used:**
- Postman: For API request execution and testing.
- Swagger UI: For technical documentation and endpoint mapping.

Pet Functionality: Pet Management
Validate the complete pet management flow (Create, Read, Update, Delete) of the API, ensuring that the basic operations work as expected.

Prerequisites:
- The Pet Store API must be online and responsive to requests.
- Postman must be configured with the Base URL (https://petstore.swagger.io/v2) to send requests.

Category	ID	Test Name	Objective	Precondition	Steps	BDD (Gherkin Syntax)	Expected Result	Actual Result (Occurrence)	Status	Priority	Severity
Happy Path	TC01	List Pets by Status	Verify the core functionality of the GET /pet/findByStatus endpoint, ensuring it returns pets with an 'available' status and that data integrity is maintained.	API is up and accessible.	1. Send GET /pet/findByStatus?status=available.	Given the API is accessible When a GET request is sent to /pet/findByStatus with the parameter 'status=available' Then the response status code should be 200 OK And the response body should be a JSON array And every pet object in the array should have the status field equal to 'available'	Status 200, response body contains pets with status = available.	As expected.	Passed	High	Major
Happy Path	TC02	Create New Pet	Validate the successful creation of a new pet using valid data, ensuring the returned object matches the data sent in the request.	API is up and accessible.	1. Send POST /pet with valid JSON (id, name, status)	Given a valid pet object is prepared with an ID, name, and status When a POST request is sent to the '/pet' endpoint with the pet object as the body Then the response status code should be 200 OK or 201 Created And the response body should be a JSON object And the returned pet's 'id' should match the ID sent in the request And the returned pet's 'name' should match the name sent in the request And the returned pet's 'status' should match the status sent in the request	Status 200/201, body contains new pet data.	Status 200 ok, pet created.	Passed	High	Critical
Happy Path	TC03	Get Pet by ID	Verify the GET /pet/{id} endpoint correctly retrieves the details of an existing pet, matching the data created in a previous test case.	A pet has been successfully created via the POST /pet request (TC02).	1. Send a GET request to /pet/{id} using the ID of the pet created in TC02.	Given a pet with a known ID exists in the system When I send a GET request to the '/pet/{id}' endpoint using that ID Then the response status code should be 200 OK And the response body should be a JSON object And the returned pet's ID should match the requested ID And the pet's name and status should match the values originally created	Status 200, body contains expected pet.	As expected.	Passed	High	Critical
Happy Path	TC04	Update Pet Details	Verify that the PUT /pet endpoint correctly updates an existing pet and that the response reflects the new data.	A pet has been successfully created via the POST /pet request (TC02).	1. Send a PUT request to /pet with a valid JSON body containing the pet's ID and new details.	Given a pet with a known ID exists in the system And a new pet object with updated data is prepared When I send a PUT request to the '/pet' endpoint with the updated pet object as the body Then the response status code should be 200 OK And the response body should be a JSON object And the returned pet's name and status should match the updated values	The response status code should be 200 OK. The response body should be a JSON object. The returned pet's data (e.g., name, status) should match the new details sent in the request.	As expected.	Passed	Medium	Major
Happy Path	TC05	Delete an Existing Pet	Verify that the DELETE /pet/{id} endpoint successfully removes a pet and that the pet can no longer be retrieved, ensuring the data is permanently removed.	A pet has been successfully created via the POST /pet request (TC02).	1. Send a DELETE request to /pet/{id} using the pet ID created in TC02. 2. Send a follow-up GET request to /pet/{id} using the same ID.	Given a pet with a known ID exists in the system When I send a DELETE request to the '/pet/{id}' endpoint using that ID Then the response status code should be 200 OK And the response body should contain a success message And a subsequent GET request to '/pet/{id}' should return a 404 Not Found status	The DELETE request should return a 200 OK status. The subsequent GET request should return a 404 Not Found status, confirming the pet no longer exists.	As expected.	Passed	Medium	Major
Negative	TC06	Get Nonexistent Pet	Verify that the GET /pet/{id} endpoint correctly handles a request for a nonexistent pet, returning a 404 Not Found status code and an appropriate error message.	The API is available and there is no pet with the specified ID.	1. Send a GET request to /pet/{id} using an ID that is not in the system.	Given the API is accessible And a pet ID that does not exist in the system is used When I send a GET request to the '/pet/{id}' endpoint using that ID Then the response status code should be 404 Not Found And the response body should contain a clear error message	Status 404, error message "Pet not found".	As expected.	Passed	Medium	Major
Negative	TC07	Create Pet Missing 'name' Field	Verify the API correctly rejects a request with a missing required field.	API is up and accessible.	1. Prepare a POST request body for a new pet, but omit the name field. 2. Send the request to the /pet endpoint with the incomplete body.	Given a request body is prepared with all required fields except for the 'name' When I send a POST request to the '/pet' endpoint with this incomplete body Then the API should reject the request with a 400 Bad Request status And the response body should contain a validation error message	Status 400 Bad Request, with a clear validation error.	Status 200 OK. Pet was created successfully.	Failed	High	Major
Negative	TC08	Create Pet with Empty 'name' Field	Verify the API correctly rejects a request when a required field has an invalid value.	API is up and accessible.	1. Prepare a POST request body for a new pet, including the name field, but leaving the value empty. 2. Send the request to the /pet endpoint.	Given a request body is prepared with the pet's 'name' field having an empty value When I send a POST request to the '/pet' endpoint with this request body Then the API should reject the request with a 400 Bad Request status And the response body should contain a validation error message	Status 400 Bad Request, with a "bad input" validation error.	As expected.	Passed	High	Major

Pet Functionality: Pet Management
Validate the complete pet management flow (Create, Read, Update, Delete) of the API, ensuring that the basic operations work as expected.

Prerequisites:
- The Pet Store API must be online and responsive to requests.
- Postman must be configured with the Base URL (https://petstore.swagger.io/v2) to send requests.

Category	ID	Test Name	Objective	Precondition	Steps	BDD (Gherkin Syntax)	Expected Result	Actual Result (Occurrence)	Status	Priority	Severity
Negative	TC09	Create or Update Pet	Validate that the PUT /pet endpoint creates a new pet when a nonexistent ID is provided.	The API is available and there is no pet with the specified ID.	1. Prepare a PUT request body with an ID that does not exist in the system. 2. Send the request to the /pet endpoint.	Given a pet ID that does not exist in the system is used And a request body is prepared with pet details for the new pet When I send a PUT request to the '/pet' endpoint with the nonexistent pet's ID Then the response status code should be 200 OK And the response body should contain the newly created pet object	Status 200 OK. A new pet with the provided ID is created successfully.	Status 200 OK. A new pet was created successfully.	Passed	Medium	Major
Negative	TC10	Delete Nonexistent Pet	Verify that the DELETE /pet/{id} endpoint correctly handles a request to delete a nonexistent pet, returning a 404 Not Found status code and an appropriate error message.	The API is available and there is no pet with the specified ID.	1. Send a DELETE request to /pet/{id} using an ID that is not in the system.	Given a pet ID that does not exist in the system is used When a DELETE request to the '/pet/{id}' endpoint using that ID is sent Then the response status code should be 404 Not Found And the response body should contain a clear error message	Status 404, error message.	As expected.	Passed	Medium	Major
Boundary	TC11	Create Pet - Name Field Boundary	Validate that the POST /pet endpoint accepts a name with up to 500 characters and returns a validation error for a name with 501 characters.	API is up and accessible.	1. (Note: The maxLength for the name field is not in the documentation. We will use 500 characters for this test.) 2. Prepare a POST request body with the name field containing a string of exactly 500 characters. 3. Send the request to the /pet endpoint. 4. Prepare a new request body with the name field containing a string of exactly 501 characters. 5. Send this second request to the /pet endpoint.	Given a pet name at both the maximum allowed length and a length that exceeds it When I send POST requests to the /pet endpoint with these names Then the response for the 500-character name should be 200 OK And the response for the 501-character name should be 400 Bad Request.	The request with 500 characters should return 200 OK. The request with 501 characters should return 400 Bad Request.	The request with 500 characters returned 200 OK. The request with 501 characters returned 400 Bad Request.	Passed	Medium	Minor
Boundary	TC12	Create Pet with Duplicate ID	Verify that the POST /pet endpoint correctly handles an attempt to create a pet with a duplicate ID, returning a 409 Conflict status and an appropriate error message.	A pet with a known ID has been successfully created via the POST /pet request (TC02).	1. Prepare a POST request body using the same ID as the pet created in TC02. 2. Send the request to the /pet endpoint.	Given a pet with a known ID already exists in the system When I send a POST request to the '/pet' endpoint with the same ID Then the response status code should be 409 Conflict And the response body should contain a clear error message indicating a duplicate entry	Status 409, "already exists".	400 Bad Request	Failed	High	Major
Boundary	TC13	Create Pet with Special Characters in Name	Verify that the POST /pet endpoint correctly rejects a pet name containing special characters.	The API is available and the validation rules for pet names are known.	1. Prepare a POST request body with a pet name containing special characters (e.g., "name": "Fluffy-@!_#$"). 2. Send the request to the /pet endpoint.	Given a pet name that contains special characters When I send a POST request to the /pet endpoint Then the response status code should be 400 Bad Request	The API should return a 400 Bad Request status.	The API returned a 400 Bad Request status.	Passed	Low	Minor

Store Functionality: Order Management
Validate the order flow and inventory tracking, ensuring that users can place orders and check stock levels correctly.

Prerequisites:
- The Pet Store API must be online and responsive to requests.
- Postman must be configured to send requests to the API's URL.

Category	ID	Test Name	Objective	Precondition	Steps	BDD (Gherkin Syntax)	Expected Result	Actual Result (Occurrence)	Status	Priority	Severity
Happy Path	TC14	Place a New Order	Validate the successful placement of a new order with valid data.	A pet must exist in the inventory.	1. Send a POST request to /store/order with a valid pet ID.	Given a valid pet ID and order details When a user sends a POST request to place a new order Then the order should be placed successfully and the system should return a successful response.	Status 200 OK, response body contains the new order's data, including id.	As expected.	Passed	High	Critical
Happy Path	TC15	Get Order by ID	Validate the retrieval of an existing order by its ID.	An order must have been created in TC01.	1. Send a GET request to /store/order/{orderId} using the ID from TC01.	Given an existing order ID When a user requests the details for that order Then the system should return the correct order data	Status 200 OK, body contains the expected order details.	The GET request returned a 404 Not Found status.	Failed	High	Critical
Negative Path	TC16	Find Order with Invalid ID	Validate that the API handles requests for an order that does not exist.	API is up and accessible.	1. Send a GET request to /store/order/{orderId} with a non-existent ID.	Given a non-existent order ID When a user request the details for that order Then the system should return a 'not found' error	Status 404 Not Found, with an error message.	As expected.	Passed	High	Major
Negative Path	TC17	Create Order with Invalid Pet ID	Validate that the API rejects an order if the pet ID is invalid or does not exist.	API is up and accessible.	1. Send a POST request to /store/order with an invalid petId.	Given a request with an invalid or non-existent pet ID When a user tries to place an order Then the system should return an appropriate error message, such as 'bad request' or 'not found', to indicate the failure	Status 400 Bad Request or 404 Not Found, with an error message.	As expected.	Passed	High	Major
Negative Path	TC18	Delete Order that Does Not Exist	Validate that the API gracefully handles the deletion of a non-existent order.	API is up and accessible.	1. Send a DELETE request to /store/order/{orderId} with a non-existent ID.	Given a non-existent order ID When a user requests the deletion of that order Then the system should return a 'not found' error	Status 404 Not Found, with an error message.	As expected.	Passed	Medium	Minor

User Functionality: User Management
Validate the complete user management flow (Create, Read, Update, Delete) of the API, ensuring that the basic operations work as expected.

Prerequisites:
- The Pet Store API must be online and responsive to requests.
- Postman must be configured to send requests to the API's URL.

Category	ID	Test Name	Objective	Precondition	Steps	BDD (Gherkin Syntax)	Expected Result	Actual Result (Occurrence)	Status	Priority	Severity
Happy Path	TC19	Create User with valid data	Validate creation of a new user with valid data	API is up and accessible.	1. Send POST /user with valid JSON (id, username, password)	Given the API is accessible, When a POST request is sent to /user with a valid JSON body Then the response status code should be 200 OK And the response body should contain the new user's data.	Status 200, body contains new user data.	As expected.	Passed	High	Critical
Happy Path	TC20	User Login with valid credentials	Validate successful login with existing user credentials.	User created in TC14.	1. Send GET /user/login with valid username and password.	Given an existing user with valid credentials When a GET request is sent to /user/login with those credentials Then the response status code should be 200 OK And the response body should confirm a successful login.	Status 200, login success message.	As expected.	Passed	High	Critical
Happy Path	TC21	Get User by Username	Validate retrieval of an existing user's details.	User logged in in TC15.	1. Send GET /user/{username} using the logged-in username.	Given an existing user When a GET request is sent to /user/{username} Then the response status code should be 200 OK And the response body should contain the user's details.	Status 200, body contains expected user details.	As expected.	Passed	High	Critical
Happy Path	TC22	Update User Details	Validate the successful update of an existing user's information.	An existing user has been created (e.g., 'mimi').	1. Send PUT /user/{username} with the existing username and a JSON body containing updated details (e.g., new email or phone).	Given an existing user with known details When a PUT request is sent to /user/{username} with a new JSON body Then the user's details should be updated successfully	Status 200, response body confirms the update, and a subsequent GET request to the same username confirms the new details.	As expected.	Passed	High	Critical
Happy Path	TC23	Delete User by Username	Validate the successful deletion of an existing user.	An existing user has been created.	1. Send DELETE /user/{username} using the existing username.	Given an existing user When a DELETE request is sent to /user/{username} Then the user should be deleted successfully	Status 200, a subsequent GET request to the same username returns a 404 Not Found error.	As expected.	Passed	High	Critical
Negative Path	TC24	Login with invalid password	Validate that login fails with an incorrect password.	User created.	1. Send GET /user/login with the correct username but an incorrect password.	Given an existing user When a GET request is sent to /user/login with an invalid password Then the response status code should be 400 Bad Request And the response body should contain an error message.	Status 400, error message about invalid credentials.	The API returned a 200 OK status code. The login was successful, and the response body contained the user's data, despite the incorrect password.	Failed	High	Critical
Negative Path	TC25	Get user with non-existent username	Validate that the API returns a 'not found' error for a non-existent user.	API is up and accessible.	1. Send GET /user/{username} using a non-existent username.	Given a non-existent username When a GET request is sent to /user/{username} Then the response status code should be 404 Not Found And the response body should contain an error message.	Status 404, "User not found" error.	As expected.	Passed	High	Major
Negative Path	TC26	Create user with duplicate username	Validate that the API prevents the creation of users with the same username.	User created in TC14.	1. Send POST /user with the same username as a previously created user.	Given an existing user When a POST request is sent to /user with a duplicate username Then the response status code should be 409 Conflict (or 400 Bad Request) And the response body should contain an error message	Status 409 or 400, error message about duplicate username.	The API returned a 200 OK status code. However, the operation did not result in a new user. A subsequent GET request for the username "mimi" returned only the original user, confirming that the duplicate request was accepted but the data was not processed.	Failed	High	Major